

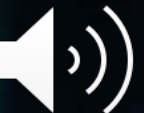


COM3021/COMM115

Data Science at Scale

Data Models

DR HUGO BARBOSA



Agenda

- Last week

How we can build efficient systems for applications that require large amounts of data?

- This week

How we can efficiently model and store such data?

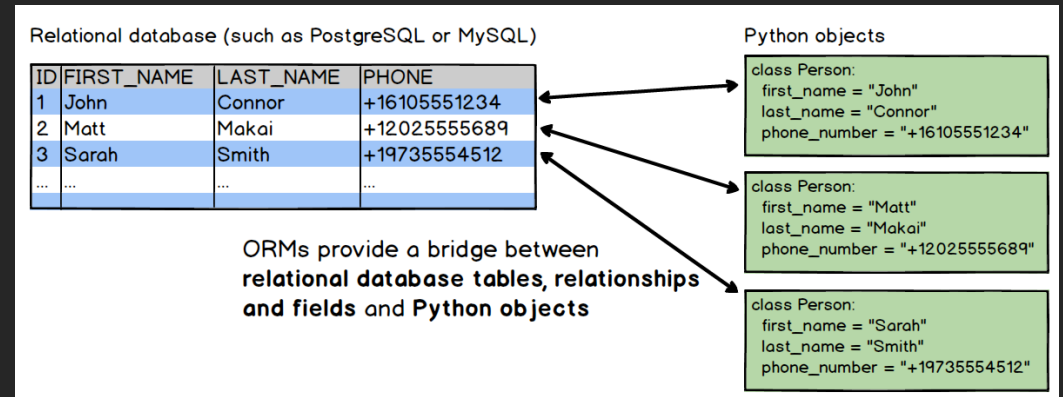
Data models

- When we build applications, we oftentimes want to represent real-world entities (e.g., people, transactions, conversations) in terms of objects and data structures
 - The storage of these objects will require them to be represented in terms of a data model (e.g., tables in an RDBM, JSON or spreadsheets).
 - A data model provides the mapping between an application entity (e.g., a Python object) and how it will be stored in a system (e.g., tables, attributes).
-

The relational model

- Object-relational models (ORMs) provide a way to map **classes** (from object-oriented systems) to a schema (e.g., tables, relations)

This translation is not always very intuitive from the application and business perspective.



Object-relational model

source: Full Stack Python

The relational model

- Modelling a LinkedIn-like data

<http://www.linkedin.com/in/williamhgate>



Bill Gates

Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience

Co-chair • Bill & Melinda Gates Foundation
2000 – Present

Co-founder, Chairman • Microsoft
1975 – Present

Education

Harvard University
1973 – 1975

Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: @BillGates



Bill Gates

Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience

Co-chair • Bill & Melinda Gates Foundation
2000 – Present

Co-founder, Chairman • Microsoft
1975 – Present

Education

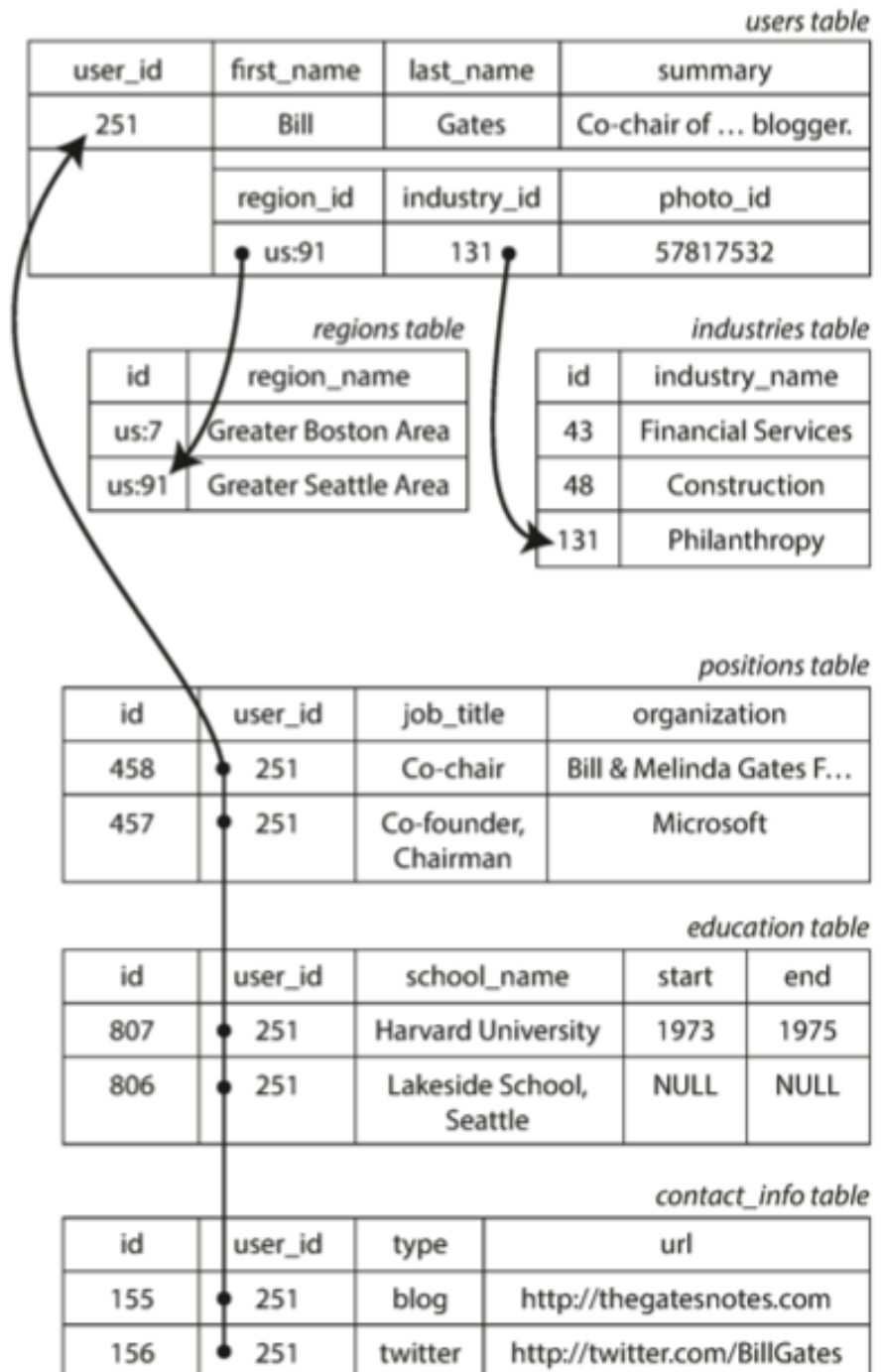
Harvard University
1973 – 1975

Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: @BillGates

Source: Martin Kleppmann - Designing Data-Intensive Applications. The Big Ideas Behind Reliable, Scalable and Maintainable Systems (2017, O'Reilly)



One-to-many relationships

- Position, Education and Contact fields
 - Normalised representation with separate tables with a foreign key to the User table
 - A field with a structured data type (e.g., XML or JSON) within the User table
 - Feature added to more recent versions of SQL*
 - Support querying and indexing on those fields*
 - Store them as JSON or XML within a text field in the DB
-



Bill Gates

Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience

Co-chair • Bill & Melinda Gates Foundation
2000 – Present

Co-founder, Chairman • Microsoft
1975 – Present

Education

Harvard University
1973 – 1975

Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: @BillGates

```
{
  "user_id": 251,
  "first_name": "Bill",
  "last_name": "Gates",
  "summary": "Co-chair of the Bill & Melinda Gates... Active blogger.",
  "region_id": "us:91",
  "industry_id": 131,
  "photo_url": "/p/7/000/253/05b/308dd6e.jpg",
  "positions": [
    {"job_title": "Co-chair", "organization": "Bill & Melinda Gates Foundation"},
    {"job_title": "Co-founder, Chairman", "organization": "Microsoft"}
  ],
  "education": [
    {"school_name": "Harvard University", "start": 1973, "end": 1975},
    {"school_name": "Lakeside School, Seattle", "start": null, "end": null}
  ],
  "contact_info": {
    "blog": "http://thegatesnotes.com",
    "twitter": "http://twitter.com/BillGates"
  }
}
```

Source: Martin Kleppmann - Designing Data-Intensive Applications. The Big Ideas Behind Reliable, Scalable and Maintainable Systems (2017, O'Reilly)

The document model

- Documents encapsulate and encode data in a semi-structured standard (e.g., XML, JSON)
 - Documents offer a much more natural translation from **objects**
All information of an object stored in a single instance in the DB
 - **More flexible**
Not required to adhere to a rigid schema
Documents within a single store can be different from one another
 - Examples: MongoDB, CouchDB (Apache SF), Firestore (Google)
-

Many-to-(one/many) relationships

- Document models can struggle with many-to relationships.

e.g., Connections and Experiences

- The information comes from the entity page (e.g., logo, name, profile photo)

- Many document DBMs implement some “join” functionality

Frequently more complicated and less efficient than SQL

Experience

**Founder**
Breakthrough Energy
2015 – Present · 6 yrs

**Co-chair**
Bill & Melinda Gates Foundation
2000 – Present · 21 yrs

**Co-founder**
Microsoft
1975 – Present · 46 yrs

Relational vs Document data model

- Advantages of document-like model:
 - *Schema flexibility*
 - *Locality*
 - *Can offer a closer representation of the data structures used by the application*
 - Advantages of relational model:
 - *Better support for joins and many-to relationships*
-

Schema flexibility

- Most document databases do not enforce any schema on the data
 - No schema means arbitrary keys and values can be added
 - *No guarantee what fields a document may contain*
-

Schema flexibility

- Most document databases do not enforce a schema

Schema-on-read: the structure is implicit and defined when the data is read by the application

Similar to dynamic typing in programming languages

Allow more efficient changes in the format of the data

Schema flexibility

- You store user's full name in one field,
Now you want to store the first name and last name separately
- Schema-on-read: In a document database, you could just make a small change to the application when the data is read:

```
if (user && user.name && !user.first_name) {  
    // Documents written before Dec 8, 2013 don't have first_name  
    user.first_name = user.name.split(" ")[0];  
}
```

- A relational database would require you to alter the DB schema:

```
ALTER TABLE users ADD COLUMN first_name text;  
UPDATE users SET first_name = split_part(name, ' ', 1);      -- PostgreSQL  
UPDATE users SET first_name = substring_index(name, ' ', 1); -- MySQL
```

Schema flexibility

- In relational DBs, schema changes can be slow and require downtime

Running UPDATE on entire tables will be slow if the table is large

You can always set values to NULL and update when you read the data

- **Schema-on-read** can be useful when:

Your data is heterogenous (i.e., different object types in the same collection)

You don't have control over the structure of the data

e.g., Tweets

Locality

- A document is usually stored in a single continuous string like JSON, XML or BSON

Pro: all the data for a given object is stored together.

- If your application often needs to access large parts of the data, it is better than storing it across multiple tables.

Con: all the data for a given object is stored together.

- When you request a document, the DB will load the entire document

If your documents are large, it can be wasteful

Locality

- Not exclusive to document DBs

e.g. Google Spanner offers locality in a relational data model

Column-family concept used in wide-column DBs (e.g., Cassandra) will manage locality in similar ways

Document and relational convergence

- Relational and document databases are becoming more similar

Modern relational DBs are incorporating document-like capabilities:

- Support for XML or JSON documents
 - PostgreSQL (> 9.3), MySQL (>5.7)
- Ability to index and query inside documents

Document databases supporting relational-like joins:

- RethinkDB
-

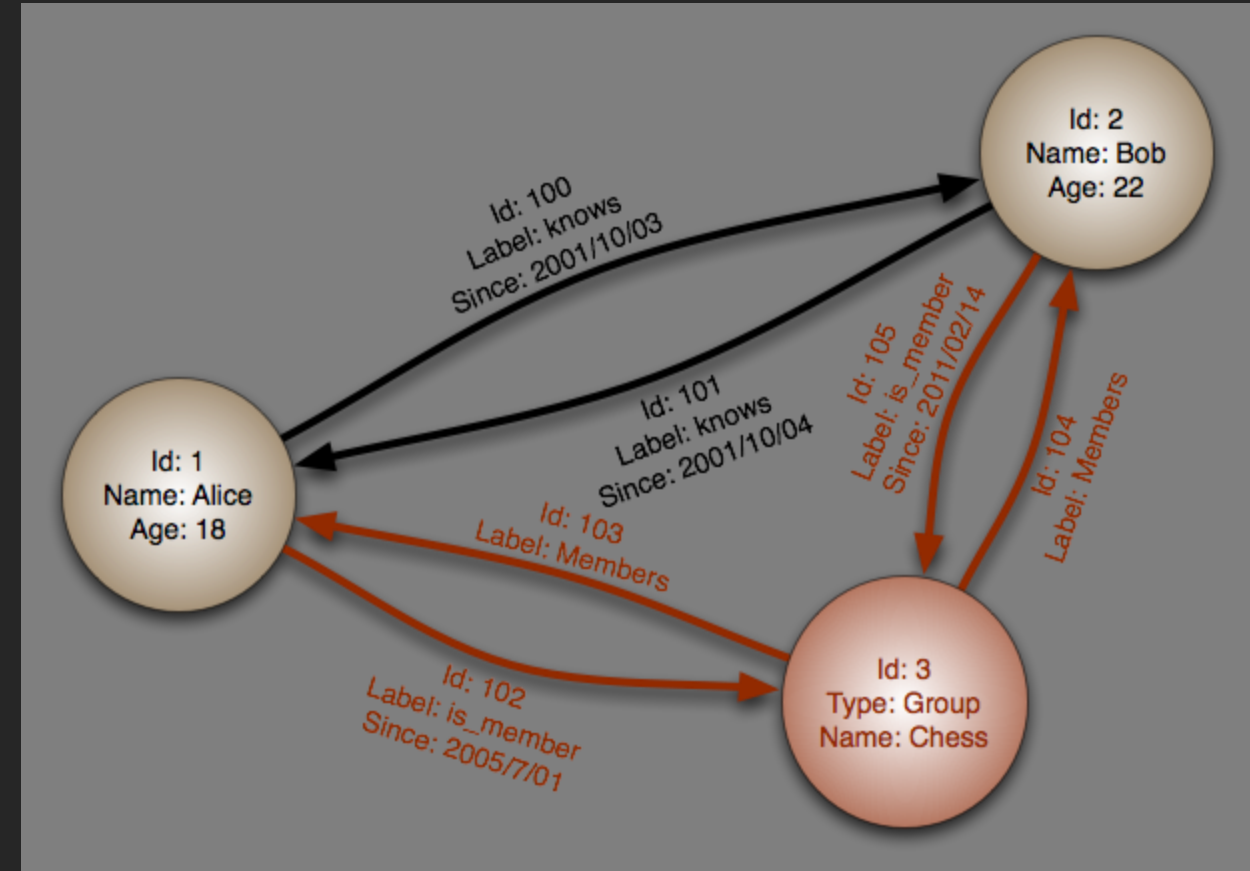
Graph-like models

- Complex many-to-many relationships are specially challenging for document and relational models
- In these cases, graph models might be more suitable

Entities represented as nodes and the relationships as edges

Examples:

- Social Networks
 - Heterogeneous nodes and edges*
- WWW
- Road infrastructure



Source: Wikimedia Foundation

Graph-like models

- Well-known algorithms can operate on graphs
 - *Dijkstra to calculate shortest routes on a road network*
 - *PageRank to calculate the relevance of a web page in a search results*
-

Graph-like models

- Property graphs

Each vertex contains

- *Unique ID*
- *Incoming edges*
- *Outgoing edges*
- *Collection of properties (key,value)*

Each edge contains

- *Unique ID*
 - *Head vertex*
 - *Tail vertex*
 - *Relationship label*
 - *Collection of properties (key,value)*
-

Graph-like models

- Can be seen as two relational tables, one for edges and one for vertices
- Any vertex can be connected to any other
- Given a vertex you can efficiently find its incoming and outgoing edges, making it easy to traverse the graph
- By using different edge labels you can represent many different relationships

```
CREATE TABLE vertices (  
    vertex_id integer PRIMARY KEY,  
    properties json  
);  
  
CREATE TABLE edges (  
    edge_id integer PRIMARY KEY,  
    tail_vertex integer REFERENCES vertices (vertex_id),  
    head_vertex integer REFERENCES vertices (vertex_id),  
    label text,  
    properties json  
);  
  
CREATE INDEX edges_tails ON edges (tail_vertex);  
CREATE INDEX edges_heads ON edges (head_vertex);
```


NoSQL databases

NoSQL databases

- Non-relational storage and retrieval of data.

- Focus on

Simplicity of design

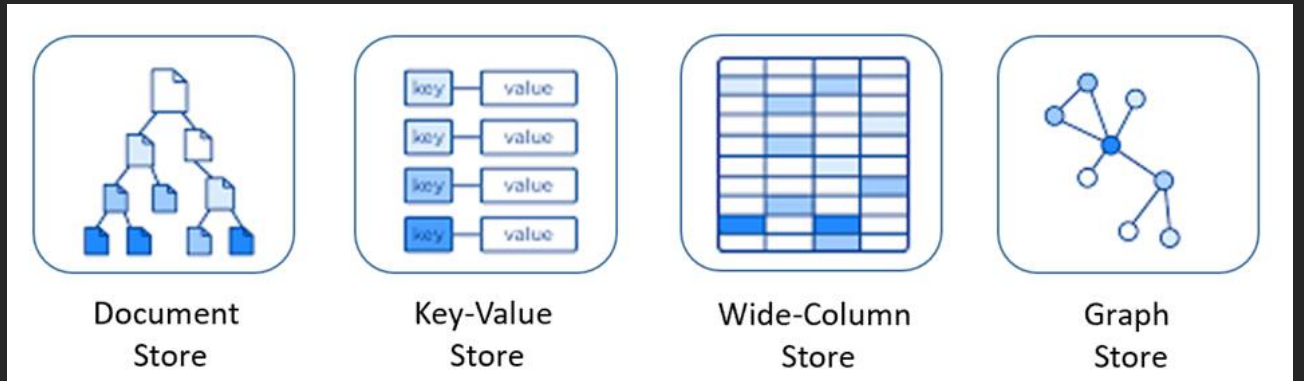
Scalability

- Horizontal scaling

Availability

NoSQL databases

- Document
- Key-value
- Wide-column
- Graph



Document store

- Different document-oriented databases will group documents in different ways

MongoDB:

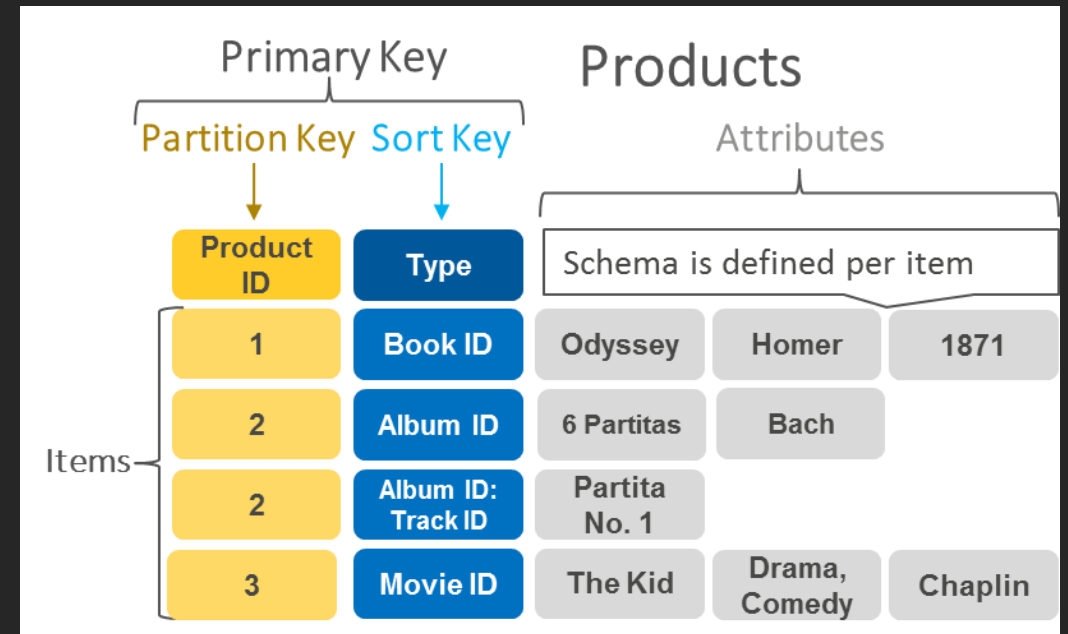
- Databases
- Collections

Equivalent to tables in RDBMs but without a rigid schema

- A query language to interact with the DBs
-

Key-value store

- Data is stored as associative arrays
- Every object is represented by a set of key-value pairs



Example of a complex primary key used by Dynamo DB
Source: Amazon AWS

Wide-column store

- Similar to relational databases

Data is modelled in tables, rows and columns

- Names and formats of the columns can vary from record to record
-